Containerising a Solana v1.18.25 Local Testbed for Reproducible Dataset Generation on Legacy x86_64 CPUs

Oleksandr Khoshaba, Ph.D.
Vinnitsia National Technical University, Ukraine
Oleksandr.Khoshaba@gmail.com
ORCID: 0000-0001-5375-6280

Version 0.1
Article DOI: 10.5281/zenodo.20098291
Software DOI: 10.5281/zenodo.20095384

Abstract

This technical report describes the design, implementation, and validation of a containerised local Solana v1.18.25 testbed for reproducible dataset generation and future latency/performance research. The work converts an initially virtual-machine-based workflow into a Podman/Docker Compose architecture with separated validator and wallet-bootstrap responsibilities. A central engineering issue addressed in this report is the execution of Solana local validator software on legacy x86_64 hardware without AVX2 support. Standard prebuilt Solana binaries and the official Solana container image were found unsuitable on an Intel Core i7-3667U CPU, where execution failed with illegal-instruction or segmentation-fault symptoms. To address this limitation, Solana v1.18.25 was built from source with AVX2 disabled and packaged into a no-AVX2 compatibility container image. The resulting testbed was validated through JSON-RPC health checks and wallet funding operations, and the runtime images were published on Docker Hub for reproducible deployment. This report documents the architecture, build workflow, Docker images, compatibility constraints, validation procedure, limitations, and planned extensions, including Geyser/Yellowstone integration and a containerised Solana latency-monitoring component.

Keywords: Solana; testbed; containerisation; Podman; Docker Compose; Docker Hub; no-AVX2; legacy x86_64; reproducibility; dataset generation; blockchain performance.

Contents

1 Introduction

Local blockchain testbeds are useful for controlled experimentation, dataset generation, and performance instrumentation. In the context of Solana research, a local `solana-test-validator` instance can provide a private and repeatable environment for evaluating transaction submission, RPC behaviour, account state updates, and latency-monitoring pipelines without interacting with a public cluster.

The original implementation used a virtual machine on which the validator, wallet funding commands, and monitoring framework were started manually. Although this setup was functional, it had several limitations:

- the environment was not fully reproducible;
- the validator process, wallet bootstrap logic, and monitoring framework were not isolated as independent services;
- deployment on another machine required manual reconstruction of the software environment;
- the architecture was not yet prepared for scaling, composition, or automated testing;
- older x86_64 CPUs without AVX2 support could not reliably execute standard Solana prebuilt binaries.

This report documents the first stage of restructuring the testbed into a containerised architecture. The implementation uses Podman/Docker Compose and separates the system into a validator service, a one-shot wallet-bootstrap service, and future monitoring services.

2 Original VM-Based Workflow

The initial workflow on the virtual machine consisted of four manually executed steps.

First, the local Solana test validator was started with a local ledger, RPC port, bind address, and optional Geyser plugin configuration:

```
solana-test-validator \
  --ledger ~/solana-localnet/ledger \
  --reset \
  --bind-address 127.0.0.1 \
  --rpc-port 8899 \
  --geyser-plugin-config ~/yellowstone-geyser.json
```

Second, a predefined payer wallet was funded using the Solana command-line interface:

```
PAYER=6avCzMrjUDebRYtSoQ6GPQENjoxDaD2Udik8JzRnKbtb
solana config set --url http://127.0.0.1:8899
solana balance "$PAYER"
solana airdrop 10000 "$PAYER"
solana balance "$PAYER"
```

Third, the performance-monitoring framework was launched:

```
cd ~/solana-latency-research
go run . --config configs/config.local.yaml
```

Finally, SSH local port forwarding exposed the VM-hosted testbed to the wider system:

```
ssh -N -o ExitOnForwardFailure=yes \
  -L 127.0.0.1:8899:127.0.0.1:8899 \
  -L 127.0.0.1:9464:127.0.0.1:9464 \
  -L 127.0.0.1:10000:127.0.0.1:10000 \
  s
```

The first containerised release focuses on reproducing the validator and wallet-bootstrap parts of this workflow. The monitoring framework and Geyser/Yellowstone integration are reserved for later releases.

3 Design Goals

The first-stage design was guided by the following goals:

1. **Reproducibility**: the testbed should be redeployable from a GitHub repository and published container images.
2. **Separation of concerns**: validator execution and wallet bootstrapping should be separate services.
3. **Legacy hardware compatibility**: the system should support at least one older x86_64 CPU without AVX2 support.
4. **Controlled exposure**: RPC and future Geyser/metrics ports should be bound to localhost on the host machine.
5. **Future extensibility**: the architecture should later support Geyser/Yellowstone and a monitoring container.
6. **Citable software**: the software release should be archived and assigned a DOI.

4 Containerised Architecture

The containerised architecture separates the local testbed into independent service roles.

4.1 Validator Service

The validator service runs `solana-test-validator`. In the containerised setup, the validator binds to `0.0.0.0` inside the container so that other services in the Compose network can access it. Host-level access remains restricted by binding published ports to `127.0.0.1`.

The validator service exposes:

- `8899/tcp`: Solana JSON-RPC endpoint;
- `10000/tcp`: reserved for future Yellowstone/Geyser gRPC access.

4.2 Wallet Bootstrap Service

The wallet-bootstrap service is implemented as a one-shot container. Its purpose is to:

- configure the Solana CLI endpoint;
- wait for the validator RPC endpoint to become available;
- check the payer wallet balance;
- request a local airdrop;
- print the final balance.

The funded payer address used in the first release is:

```
6avCzMrjUDebRYtSoQ6GPQENjoxDaD2Udik8JzRnKbtb
```

The initial funded amount is:

```
10000 SOL
```

4.3 Monitoring Service Placeholder

A future service named `monitor` is reserved for the Solana latency-monitoring framework. This component is not included in the functional scope of the v0.1.x software release. Its expected responsibilities include latency measurement, metrics exposition, and dataset generation.

5 CPU Compatibility Problem

During validation on an Intel Core i7-3667U host, the standard prebuilt Solana v1.18.25 binaries failed to execute correctly. The host CPU supports AVX but does not support AVX2:

```
CPU: Intel Core i7-3667U
Architecture: x86_64
AVX: yes
AVX2: no
SSE4.1: yes
SSE4.2: yes
```

Observed symptoms included:

```
Illegal instruction
Aborted
solana-test-validator exited with code 139
general protection fault
```

The official `solanalabs/solana:v1.18.25` image was also unsuitable on this host, confirming that the issue was not caused by the custom container wrapper but by the binary/runtime compatibility of the prebuilt Solana release on this hardware.

6 Source-Built no-AVX2 Compatibility Image

To address the compatibility issue, Solana v1.18.25 was built from source with AVX2 disabled. The resulting image was used as the base for runtime validator and wallet-bootstrap images.

The development source image is:

```
localhost/solana-source-noavx2:v1.18.25
```

The runtime images are:

```
localhost/solana-localnet-validator:v1.18.25-noavx2
localhost/solana-wallet-init:v1.18.25-noavx2
```

The published Docker Hub images are:

```
docker.io/khoshaba/solana-source-noavx2:v1.18.25-noavx2-ivybridge
docker.io/khoshaba/solana-localnet-validator:v1.18.25-noavx2-ivybridge
docker.io/khoshaba/solana-wallet-init:v1.18.25-noavx2-ivybridge
```

The `ivybridge` suffix is intentional. The current compatibility image was built and validated on an Ivy Bridge CPU and should not be described as a universal x86_64 image until tested on a wider hardware matrix.

7 Docker Compose Workflow

The release workflow uses `compose.release.yaml`. This file references Docker Hub images rather than building local images.

A release-mode startup is performed with:

```
podman compose -f compose.release.yaml up
```

The release Compose file includes:

- a long-running validator container;
- a one-shot wallet-init container;
- a persistent ledger volume;
- localhost-bound port exposure;
- a validator healthcheck.

The release-mode RPC endpoint is:

```
http://127.0.0.1:8899
```

8 Validation Procedure

8.1 RPC Health Check

The validator RPC endpoint is checked using JSON-RPC:

```
curl -s http://127.0.0.1:8899 \
  -H "Content-Type: application/json" \
  -d '{"jsonrpc":"2.0","id":1,"method":"getHealth"}'
```

The expected response is:

```
{"jsonrpc":"2.0","result":"ok","id":1}
```

8.2 Payer Balance Check

After the wallet bootstrap container completes, the payer balance is checked with:

```
podman exec -it solana-localnet-validator \
  solana --url http://127.0.0.1:8899 \
  balance 6avCzMrjUDebRYtSoQ6GPQENjoxDaD2Udik8JzRnKbtb
```

The expected result is:

```
10000 SOL
```

9 Validation Results

Table 1: Initial validation matrix.

Machine	CPU	AVX	AVX2	Health check	Payer balance
think	Intel Core i7-3667U	yes	no	ok	10000 SOL
fedora	TODO	TODO	TODO	ok	TODO

The initial validation confirms that the source-built no-AVX2 image can run a Solana v1.18.25 local validator on the tested legacy x86_64 host.

10 Software Availability

The software artifact is archived on Zenodo:

Software DOI: 10.5281/zenodo.20095384

The article/technical report DOI is:

Article DOI: 10.5281/zenodo.20098291

The Docker Hub images are:

```
docker.io/khoshaba/solana-source-noavx2:v1.18.25-noavx2-ivybridge
docker.io/khoshaba/solana-localnet-validator:v1.18.25-noavx2-ivybridge
docker.io/khoshaba/solana-wallet-init:v1.18.25-noavx2-ivybridge
```

Immutable image digests are recorded in:

```
release/v0.1.0-image-digests.txt
```

The GitHub repository is:

https://github.com/okhoshaba/solana-containerised-testbed

11 Limitations

The v0.1.x release has several limitations:

- Geyser/Yellowstone plugin runtime integration is not yet included.
- The latency-monitoring framework is not yet containerised.
- The no-AVX2 image is validated on an Ivy Bridge CPU and should not yet be treated as a universal compatibility image.
- The system is intended for local research testbed use, not production validator operation.
- The source-built image may have different performance characteristics from official prebuilt Solana binaries.

12 Future Work

Planned future work includes:

1. containerising the Solana latency-monitoring framework;
2. adding Geyser/Yellowstone plugin support;
3. publishing a broader hardware compatibility matrix;
4. producing a generic no-AVX2 build with more conservative CPU flags;
5. automating release validation through GitHub Actions or an equivalent CI system;
6. generating structured datasets and assigning dataset DOIs.

13 Conclusion

This report presented the first containerised release of a local Solana v1.18.25 testbed for dataset generation and performance research. The system separates validator and wallet-bootstrap responsibilities into distinct containers and provides a release-mode Compose workflow based on Docker Hub images. A key technical contribution is the source-built no-AVX2 compatibility image, which enables execution on a tested legacy x86_64 CPU where standard prebuilt Solana binaries fail. The resulting software release has been archived with a DOI, and the current report provides the methodological and architectural documentation needed for reproducibility and future extension.

References

[1] Khoshaba, *Solana Containerised Testbed*, Zenodo software release. DOI: 10.5281/zenodo.20095384.

[2] Khoshaba, *Containerising a Solana v1.18.25 Local Testbed for Reproducible Dataset Generation on Legacy x86_64 CPUs*, Zenodo technical report. DOI: 10.5281/zenodo.20098291.

[3] Solana Labs, *Solana Documentation and Software Releases*. https://github.com/solana-labs/solana

[4] Podman, *Podman Documentation*. https://docs.podman.io/

[5] Docker, *Docker Hub*. https://hub.docker.com/

[6] Zenodo, *Zenodo Documentation*. https://help.zenodo.org/